学生上机步骤

RSA 模数的共享素因子攻击 · COMP3040《密码学基础》· 2 学时

怎么用这份文件

每一步都是三段：

- **敲：** —— 你要执行的操作
- **该看到：** —— 对照你的屏幕。**不一样不要往下走。**
- **要记下：** —— 报告里要用到的数字，先记在纸上或单独一个文本文件

每一步开头会标 **【本节要填 N 处空】**。四个任务一共 **29 处空**（10 + 6 + 9 + 4）——全部填完，整条攻击链才跑得通。

四个任务的详细讲解在《实验指导书》§5.1–§5.4；这份文件是它的**操作版**：你先敲、先看到、先记下。

注意：本文里 [···] 表示真实输出里**要你自己跑出来才知道**的内容。不要试图猜，跑一遍就有了。

第 0 步　课前作业：环境自检

（时间：第 0–6 分钟 —— 课上这 6 分钟只用来处理「失败」的机器。）

【本节要填 0 处空】

敲：

```
cd rsa-lab4/code
python check_env.py
```

该看到：

```
======================================================================
rsa-lab4 环境自检（本科 2 学时版）
======================================================================
1. 解释器
  ✓ Python 版本 >= 3.10             当前 3.1x
  ...
9. 填空机制
  ✓ 未填的空会抛出专门异常
  ✓ except Exception 吞不掉它
  ✓ 赋值被拒绝

======================================================================
填空进度
======================================================================
  task1_math.py      ▒▒▒▒▒▒▒▒▒▒  0 / 10
  task2_pair.py      ▒▒▒▒▒▒  0 / 6
  task3_scan.py      ▒▒▒▒▒▒▒▒▒  0 / 9
  task4_decrypt.py   ▒▒▒▒  0 / 4

  合计 0 / 29

======================================================================
全部通过 —— 可以开始实验了。
======================================================================
```

要记下：

项	你的值
Python 版本	
数据文件两项检查是否全 ✓	

如果卡住： 记住**不要**加 `-I` 或 `-E` 参数。加了会报 `ModuleNotFoundError: No module named 'fill'`。 按提示去掉即可。

第 1 步　任务一：数学地基（egcd / modinv / φ）

【本节要填 10 处空 —— `task1_math.py` 的 `FILL[1]` ~ `FILL[10]`】

1.1 先看看这个文件

敲：

```
python task1_math.py          # 不加参数
```

该看到： 一段帮助信息，最后一句是「加 `--selftest` 跑自检」。

要记下： 无。

> 这一步是想让你知道：**代码骨架始终可以运行。** 填空只在执行到那一行时才失败，不影响你读代码、看帮助。

1.2 填 FILL[1] ~ FILL[4]：egcd 的四个赋值

打开 code/task1_math.py，找到 egcd 函数里的四处 TODO。

FILL[3]、FILL[4] 是**循环体里的两行同步更新。** 先读一遍骨架里已经写好的那一行：

```
old_r, r = r, old_r - q * r          ← 已经写好的第一行
```

你要填的两行**结构完全一样**，只是把 old_r/r 换成 old_s/s、 old_t/t，把 q * r 换成 q * s、q * t。

FILL[1]、FILL[2] 是**循环开始之前**的两个初值。初值来自两个平凡等式：

```
a = 1·a + 0·b        →  a 的系数是 (1, 0)
b = 0·a + 1·b        →  b 的系数是 (0, 1)
```

对照上面 (old_r, r) = (a, b) 的写法，你就知道那两行该长什么样。

改法：把 FILL[1] 这几个字**整体删掉**，在原位写你的语句。

```
FILL[1]                    → 改成 →    old_s, s = 1, 0
```

> ⚠ **不要写 FILL[1] = ...。** 会把 FILL 这个名字绑到别的东西上， 程序会明确拒绝。
>
> ⚠ **也别把初值那两行填到循环体里面。** 那会在每一轮把系数重置回初值， 循环跑完拿到的永远是 (1, 0) —— 【一】会立刻抓到。

1.3 填 FILL[5] ~ FILL[7]：modinv

同一个文件，modinv 函数。三处，都很短。

FILL[5] 是调用你刚写好的 egcd —— **一次取出三个返回值。**

FILL[6] 写的是一个**条件表达式**（不是一整行 if）—— 骨架已经写好了 if ... :，你只填括号里的条件。 这一句拦下的是「**逆元根本不存在**」的情形。

FILL[7] 是函数的 return 语句要返回的东西。

> ⚠ **返回之前必须归约。** `egcd` 给出的系数是**可能为负**的：`egcd(3, 7)` 会给 `x = -2`，而逆元的规范形式是 5。忘了归约的话，【二】照样通过 —— 因为 `3 × (-2) % 7 == 1` 成立，Bézout 也成立。**只有【三】的范围检查能抓住它。**

1.4 填 `FILL[8]` ~ `FILL[10]`：φ(n) —— 本包后果最重的一处

最后三处在 `totient` 函数。φ 原来是随包给的，现在**归你写** —— 因为「$\varphi(p^2)$ 该写成什么」恰恰是全包后果最重的那个陷阱。

两种情形要分开：

情形	公式
`p != q`	`(p - 1) * (q - 1)`
`p == q`（完全平方 $n = p^2$）	`p * (p - 1)`，**不是** $(p-1)^2$

`FILL[8]` 是 `if` 里的条件（拦下**普通**情形）；`FILL[9]`、`FILL[10]` 是两个分支各自的 `return`。

> ⚠ **为什么它后果最重：写错了不报错。** 顺手写成 `(p - 1) * (q - 1)` 的话，`p ≠ q` 时完全正确，`p == q` 时算出 $(p-1)^2$ —— 比真值 $p(p-1)$ 少了 $p-1$，一个 512 位的数。算出来的 d 是一个**合法范围内的错值**：不越界、不抛异常、程序不变慢，甚至连 $e \cdot d \equiv 1$ 都照样成立（模的是那个错的 φ）。
>
> **`verify.py` 的「d 落在 [0, φ)」检查抓不住它** —— 因为 $(p-1)^2 < p(p-1)$，错的 d 本来就更小，照样落在范围里。真正能抓住它的只有两处：【五】，和几十分钟之后任务四的往返解密。

> **手算验证**：`p = 3`、`n = 9` 时，与 9 互素的数是 1,2,4,5,7,8 共 6 个；$p(p-1) = 3 \times 2 = 6$ ✓，而 $(p-1)^2 = 4$ ✗。
>
> 两个分支**写反**了会怎样？【五】用 `totient(61, 53)` 当场验：正确的是 3120，写反会得到 3660 —— 所以这一处**错了会响**，不用担心它悄悄溜过去。

1.5 跑自检

敲：

```
python task1_math.py --selftest
```

该看到：

```
============================================================
任务一自检：扩展欧几里得 + 模逆
============================================================
【一】Bézout 恒等式  a·x + b·y = gcd(a, b)
  全部 7 例通过

【二】模逆  a · modinv(a, m) ≡ 1 (mod m)
  全部 5 例通过

【三】⚠ 返回值必须在 [0, m) 区间 —— 负 d 的唯一防线
  modinv(3, 7) = 5   通过（落在 [0, 7) 内）

【四】⚠ 逆元不存在时必须抛 ValueError，而不是返回一个错值
  全部 4 例都正确抛出了 ValueError

【五】⚠ φ(n) 的两种情形 —— 错了要到任务四才发作
  全部 3 例通过（含 (p-1)² = 36 ≠ 42 的反例）

全部通过 —— 任务一完成，可以带着它上任务二了。
```

> **【三】和【四】是「负向检查」** —— 它们检查的是 「**本该报错的时候有没有报错**」。 正着做对只证明你覆盖到的情形对； **负着不做错，才说明你写完整了。**

自检最后会打印三问。其中 (c) 就是**实验报告 §** 的那一问： $\varphi(p^2)$ 算错成 $(p-1)^2$ 时程序一声不吭 —— 为什么「d 落在 [0, φ) 内」这条检查抓不住它？现在就可以先想。

如果卡住： 见文末表格的 `FILL[1]~FILL[10]` 十行。

第 2 步　任务二：拆开一对设备

【本节要填 6 处空 —— `task2_pair.py` 的 `FILL[1] ~ FILL[6]`】

2.1 填 `FILL[1] ~ FILL[4]`：扫描

`scan_pairs` 函数里的双重循环，四处。

`FILL[1]` 是**内层循环的起点**。提示里给了两个理由 —— 想清楚再写： 无序对 `(i, j)` 和 `(j, i)` 是一回事，只该算一次；而且绝不能让自己跟自己配 `(gcd(n, n) = n,` 会被误读成「模数重复」）。

`FILL[2]` 是**求两个模数的最大公约数**。一行。

> 提示：文件开头已经 `import math` 了。**用 `math.gcd`，不要手写欧几里得** —— 实测快一个数量级（8.8 ~ 16.8 倍）。 攻击之所以可怕，一半在于数学，另一半在于它便宜到可以当日常巡检跑。

FILL[3] 是**什么样的 g 值得记下来** —— 一个条件表达式。

> ⚠ **这一处写窄了，整条链会一起错，而且全程不报错。** 提示说得很直白：现在**先把所有异常都抓出来**，怎么分类是任务三的事。 如果在这里顺手把 `g == n`（两个模数完全相同）滤掉， 那两台重复模数**永远不会浮出水面**：扫描会安静地少报一条命中， 任务三的「命中条数」和「涉及台数」跟着一起错（可分解台数碰巧不变）—— 而「你漏报了一类异常」和「语料里没有这一类」，看起来是一样的。

FILL[4] 是**把这一对记下来。**它在一个 `if` 里面， 提示写着「设备号 i、设备号 j，以及 g」—— 也就是往 `hits` 里追加一个**三元组。**

2.2 填 FILL[5]、FILL[6]：拆开这一对

`break_pair` 函数。

FILL[5] **不是一行** —— 是一整块代码，要把三种情形分开：

情形	含义	处理
`g == 1`	两台设备无关，**不是这一对**	抛 `ValueError`，说清「不共享素因子」
`g == n_a`	两个模数**完全相同** —— 异常，但**拆不开**	抛 `ValueError`，说清「模数完全相同，异常但拆不开」
`1 < g < n_a`	**共享素因子** —— 就是我们要的	`p, q = g, n_a // g`

骨架里 FILL[5] 的位置前后有注释说明这三个分支该怎么处理。 **三种都要写到**，一个都不能漏。

> **为什么 `g == n_a` 值得单独写一条**：它是本实验的其中一个点 —— **「异常」不等于「可分解」。**你发现了有设备在用同一个模数， 这是个严重的安全问题；但仅凭这份语料，拿到的是 n 本身而不是因子。 攻击者知道了「重复」，却仍然不知道因子。

> ⚠ **漏掉 `g == n_a` 那一支的后果**：程序会带着 `p = n、q = 1` 往下走， φ 变成 0，然后 `modinv` 报一个**莫名其妙的错** —— 错误信息指向任务一，而真正的问题在这里。 **错误信息在哪，问题不一定在哪。**

FILL[6] 是把公开指数 e 变成私钥 d。一行 —— 用你在任务一写的那个函数， 注意参数顺序是 `(e, φ)`，别反了。

2.3 跑

敲：

```
python task2_pair.py
```

(本机约 6~7 秒)

该看到：

```
================================================================
任务二：扫描 1000 个模数
================================================================
两两组合数：C(1000, 2) = 499,500
（本机约 6~7 秒，请稍等）

命中          设备 i     设备 j    gcd 的位长
----------------------------------------------------------------
          [···]   [···]   512 位
          [···]   [···]   512 位
          [···]   [···]   512 位
          [···]   [···]   1024 位
          [···]   [···]   512 位

命中条数：5
扫描用时：约 6 秒
已写出：...\data\hits.json（任务三会读它）
```

然后是「拆开其中一对」和「往返验证」两段，最后：

```
往返全部成功 —— 这把私钥是正确的。
```

要记下：

项	你的值
命中条数	
有几行的 gcd 位长是 512 位	
有几行是 1024 位	
被拆开的那对设备号	
它的 gcd 位长	
它的 φ(n) 位长	
它的 d 的承诺值	
往返验证是否全部 ✓	

> ⚠ **那张表里有一行的 gcd 位长是 1024 位**，和模数一样长 —— 从上往下数的第 4 行。想一想这意味着什么 —— 先想，别急着翻答案。

> ⚠ **把「条」和「台」分开想。** 表里一行是**一条命中**（一对设备）， 不是一台设备；一条命中里有**两台**设备。等任务三跑完，你会从同一批 设备里数出两个不同的数 —— 这就是那个问题的伏笔。

第 3 步　任务三：扫描 + 分类 + 恢复全部私钥【★ 主交付物】

【本节要填 9 处空 —— `task3_scan.py` 的 `FILL[1]` ~ `FILL[9]`】

本任务产出 `data/weak_devices.json` —— 这是本次实验的**主交付物**。

3.1 填 `FILL[1]` ~ `FILL[3]`：把「对」翻译成「设备」

`classify` 函数。三处。

`FILL[1]` 是一个条件表达式 —— 判断这一条命中属于哪一类。

> ⚠ **这是全包价值最高的一处空。** 条件要写成 `1 < g < moduli[i]`（g 要**严格**介于 1 和 `moduli[i]` 之间）。 写成 `g > 1` 的话，「模数重复」那一类会被并进「可分解」—— 台数从 6 变成 8，`verify.py` 会报「这些设备其实不可分解，但你列进来了」。 两个方向都错得很响，改回来也就一行 —— 想清楚再写。

`FILL[2]`、`FILL[3]` 是把**设备 i 和设备 j 都**记下来，一处一个容器。 两处写法完全一样，只是容器不同。

关键点：一条命中 `(i, j, g)` 里有**两台**设备，**两台都要记。** 容器的类型提示你注意：它是 `set` 不是 `list` —— 想清楚为什么。 （3 台设备互相共享时，它们之间会有 3 条命中：`(7,8)` `(7,9)` `(8,9)`。 用 `list` 的话，设备 7 会被数两遍，台数就偏大了。）

3.2 填 `FILL[4]`、`FILL[5]`：找完全平方

`find_perfect_squares` 函数。两处。

`FILL[4]` 是求 n 的**整数**平方根。一行。

> ⚠ **必须用 `math.isqrt`，不能用 `int(n ** 0.5)`。** n 有 1024 位，而 float 只有 53 位精度 —— `n ** 0.5` 给的是一个 完全错误的近似值，你拿它平方去比 n **永远不相等。**
>
> 而且它**不报错。**这是本实验里最贵的一类 bug —— 检测它的唯一办法，是**提前知道你应该找到几台。**

> 为什么要单独做这一步：两两 GCD 对这类设备**完全沉默。** 它的两个素因子是同一个 p，别的设备都是独立随机的， 没有任何一台会持有 p —— 扫描一条命中都不会给它。

FILL[5] 是「判断 + 记录」这一整块：一个 if 比较 r * r 和 n， 成立时把设备号 dev 追加进 out。要用**精确的整数比较。**

3.3 填 FILL[6] ~ FILL[9]：恢复私钥

recover_keys 函数，两条路。

共享素因子（FILL[6]~FILL[8]）—— 这几台设备要**跟邻居借** p：

- FILL[6]：n 和**搭档**的模数求 gcd —— 那就是共享的素数 p。 ⚠ 第二个参数必须写搭档的模数。写错的后果很阴：不会当场报错， 它会安静地把 p 算错，然后在 φ 和 modinv 里炸 —— **错误信息又指向任务一。**
- FILL[7]：n 被 p 整除，商就是另一个素因子。注意是**整除 //** —— 用真除 / 会得到一个 float（只有 53 位精度），φ 跟着变成浮点数， modinv 会报「没有逆元」，而它说的模数是一串 8.5…e+307 这样的 科学计数法：错得莫名其妙，但那个小数点暴露了一切。
- FILL[8]：把公开指数 e 变成 d。一行 —— 还是用你任务一写的那个 modinv，参数是 (e，φ)。

完全平方（FILL[9]）—— p 不用借别人，p = isqrt(n) 自己开出来。

> ⚠ **不要手写 (p - 1) * (q - 1)。** 在这条路上 p 和 q 是**同一个数**，(p−1)² 是错的（真值是 p(p−1)，两者差一个 512 位的数）。 用你在任务一写的那个 totient —— 它认得 p == q 和 p != q 两种情形， 这里要做的只是**选对分支、调用它。**
>
> 这一处错了不会有任何即时报警。任务一那个自检块存在的理由， 就是替这个「几十分钟之后才发作」的错作证 —— 要是你在任务一的 totient 里就写错了 p == q 那一支，现在就是它还债的时候。

3.4 跑

敲：

```
python task3_scan.py
```

如果它说找不到 hits.json，一条命令补上：

```
python task3_scan.py --adopt-hits
```

（它调用**你自己写的** scan_pairs 重扫一遍，约 6~7 秒，不改变任何答案。）

该看到： 五个区段 —— 命中统计、类别表、完全平方检测、 「★ 三个数，别混」、恢复私钥表。最后：

```
已写出：...\data\weak_devices.json
字节数：4,313
```

要记下：

项	你的值
命中条数	
命中涉及设备 台数	
真正可分解 台数	
完全平方找到几台	
模数重复几台	
恢复了多少把私钥	
weak_devices.json 字节数	

> **前三格必须不相等。** 如果相等，说明你有一类设备漏了或者算重了 —— 别急着改，先想：一条命中**条**和一台**设备**，是一回事吗？ 完全平方那一类，和两两扫描有什么关系？

3.5 自查

敲：

```
python verify.py
```

该看到：

```
✓ 字段齐全、类型正确
✓ 6 把私钥全部通过：d 落在 [0, ϕ)、承诺值自洽、往返加解密成立
✓ 分类自洽：可分解 / 异常不可分解的划分是正确的

性质检查全部通过。
```

确认「一个不多、一个不少」再加一层：

```
python verify.py --appendix-b
```

要记下： verify.py 是不是全 ✓。

> verify.py **不告诉你答案**，它只检查**性质。** 而且它**不读你的自述**：φ 和因子都是它从语料里**重新算一遍**的 —— 验证方不能向被验证方要证据。性质全部通过 = 你交的东西内部一致。 想确认数目对不对，才用 --appendix-b。
>
> **建议先把分析做完再跑 --appendix-b。** 数不对本身是一条线索。

第 4 步　任务四：解开那份通知

【本节要填 4 处空 —— task4_decrypt.py 的 FILL[1] ~ FILL[4]】

黑盒 decrypt_hw.py 负责与攻击无关的部分（base64、长度前缀、切 IV、 AES-256-CBC、PKCS#1 编码），直接给你。你要写的是**属于你自己的那几行** —— 整场攻击的收尾。

4.1 填 FILL[1]：私钥运算

一行。用**你自己算出的** d 做那一次模幂。

4.2 填 FILL[2]：还原成定长字节串

一行 —— 把模幂的结果还原成 EM 块。rsa_common 里已经 import 进来了 一对互逆的工具（i2osp 配 byte_len），提示里写清了要用哪一个。

> ⚠ **必须用「定长」的那一个。** 图省事写成「最短能装下就行」的话， EM 开头的 0x00 会被吃掉 —— 而 PKCS#1 要求第 1 字节**正是** 0x00。 后果是**每一把**钥匙都判为「填充不合法」， 你会顺着错误信息去怀疑 d 算错了，方向完全跑偏。

4.3 填 FILL[3]：剥掉 PKCS#1 填充

一行。留下真正的会话密钥。

> 提示：rsa_common 里已经 import 进来了一个函数， 名字以 pkcs1v15 开头、decode 结尾。

4.4 填 FILL[4]：反向确认

一行，但是最难的一处 —— 它是一个「**不成立**」的条件（成立就 return None）。

> ⚠ **要加密的是整块 em，不是里面的 session_key。**
>
> ```
> EM = 0x00 || 0x02 || PS || 0x00 || session_key
> └──────────── RSA 作用在这一整块上 ────────────┘
> ```

> 用 session_key 去加密，对一把**完全正确**的私钥也会返回 False。 你会以为 d 错了，然后去改一个本来没错的地方。
>
> 这一问就是**报告 §四** 的第 1 问。

4.5 跑

敲：

```
python task4_decrypt.py
```

该看到：

```
RSA 密文        : 1023 位
IV              : d941f224df4ef958fcd07f26a82f93b0
AES 密文体      : 16,912 字节

手上有 6 把私钥，逐个试过去：

  设备  [···] : 不是这把
  ...
  设备  [···] : ✓ 成功

已写出     : ...\data\homework.pdf
字节数     : 16,896
SHA-256    : d3c1c5b3b6bc7577ff19f6aa1a21e37ad473e9d1b6b5997450e80f2(
```

要记下：

项	你的值
哪台设备的私钥打开了它	
字节数	
SHA-256	
与附录 B 里那份明文的 SHA-256 是否逐字符相同	

> ⚠ **能打开 ≠ 做对了。**
>
> 如果你用了 `aes_soft.cbc_decrypt`（保留 PKCS#7 填充）而不是 `decrypt_hw.aes_cbc_decrypt`（去填充），会多出 1~16 字节， **PDF 阅读器照样打得开**，但字节数和 SHA-256 对不上。
>
> **必须核对 SHA-256，不是「能打开就行」。**

第 5 步　清单

交什么

#	交什么	怎么交
1	`data/hits.json`	原始文件
2	`data/weak_devices.json`	原始文件
3	`data/homework.pdf`	原始文件
4	`code/` 里你修改好的4个文件	完整代码随报告一起交
5	实验报告	Word 文件（按模板命名）

卡住了怎么办

环境和运行

现象	原因	怎么办
`ModuleNotFoundError: No module named 'fill'`	不在 `code/` 目录里跑，或用了 `python -I` / `-E`	先 `cd` 到 `code/`；去掉 `-I`、`-E`。`-I` 会把脚本目录移出 `sys.path`
`FILL[n] not filled yet …`，结尾写着**退出码 3**	那一处空还没填	**这不是程序出错**，是正常状态：按提示打开文件、翻到那一行，把 `FILL[n]` 换成你的代码再跑。（「空没填」这个信号写成了专用异常，`except Exception` 吞不掉它 —— 所以它总能找到你）
打印乱码	终端编码不是 UTF-8	`PYTHONIOENCODING=utf-8 python -X utf8 xxx.py`
`python` 开的是应用商店	Windows 的应用执行别名	用 `py -3 xxx.py`，或关掉那个别名
中文在报错里变成 `\xXX`	GBK 控制台	`fill.py` 已处理，正常不用管
手机上打不开 PDF	——	用电脑。PDF 有个中文字体依赖，手机阅读器可能显示不全

还是不行

1. 跑 `python run_all.py --progress` 看还有几个空；
2. 跑 `python check_env.py` 看环境有没有问题；
3. 看代码逻辑是不是正确，特别是容易出错的几个点。